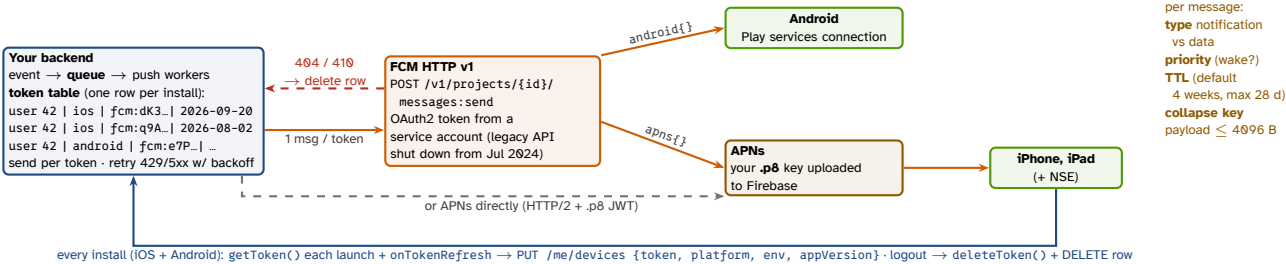


In one line: Your server addresses **installs**, not users. Each install gets a **token** (on iOS the FCM SDK maps the APNs tokens to an **FCM token**) and uploads it keyed by **user + device**; the backend **fans out** to every live token via **FCM HTTP v1** (or APNs directly) and deletes the ones reported dead. What runs on arrival depends on **message type**, **app state** and platform.



Token lifecycle

- Register:** Android — the FCM SDK gets a token. iOS — registerForRemoteNotifications → APNs token → the FCM SDK (swizzled AppDelegate, or set Messaging.messaging().apnsToken when FirebaseAppDelegateProxyEnabled = NO) → FCM token. APNs token ≠ FCM token: never send one to the other's API.
- Refresh:** reinstall, restore to a new phone, cleared app data, and at FCM's discretion → onTokenRefresh. Upload on every launch with a timestamp.
- Many per user:** one row per install (phone + tablet + old phone), each with platform, APNs environment, app version, locale. **Stale:** FCM calls a registration stale after a month offline and expires Android tokens after 270 days inactive — prune on your timestamps.
- Delete:** logout → deleteToken() + delete/unlink the row, or the next person on that phone gets the last user's pushes. FCM is moving to Firebase Installation IDs (FIDs).

Message type × app state

	foreground	background / killed
Android notification	onMessageReceived — you show it	system tray, your code not called
Android data	onMessageReceived	onMessageReceived (RN: background handler, headless JS)
iOS alert	willPresent — you choose	system shows it; NSE if mutable-content:1
iOS data-only	didReceiveRemoteNotification (RN onMessage)	content-available:1, priority 5, push-type background: throttled; none after force-quit

Priorities

Android **high** may wake a device in Doze; if high-priority messages don't produce visible notifications FCM may downgrade them to normal. APNs **10** = now (alerts), **5** = power-friendly — FCM rejects priority **10** for data-only messages to Apple devices. Android 13+: runtime POST_NOTIFICATIONS; Android 8+: every notification needs a **channel**.

Rich & encrypted payloads

iOS: apns.fcm_options.image + mutable-content → an NSE calls Messaging.serviceExtension().populateNotificationContent(_:withContentHandler:). **E2EE:** send ciphertext in data + a generic alert; the NSE decrypts with a key shared via an App Group / Keychain access group and rewrites title/body. Android: send **data** only, decrypt in your service, post the notification yourself.

Example — one FCM v1 message, both platforms

```
{
  "message": {
    "token": "fcm-token-of-this-install",
    "notification": {
      "title": "Alice",
      "body": "New message",
      "data": {
        "chatId": "42",
        "priority": "high",
        "collapse_key": "chat-42",
        "notification": {
          "channel_id": "messages"
        }
      },
      "headers": {
        "apns-priority": "10",
        "apns-collapse-id": "chat-42",
        "payload": {
          "aps": {
            "mutable-content": 1,
            "thread-id": "42"
          }
        },
        "fcm_options": {
          "image": "https://example.com/a.jpg"
        }
      }
    }
  }
}
```

```
// index.js - OUTSIDE React, before AppRegistry
messaging().setBackgroundMessageHandler(async (m) => {
  await inbox.save(m.data) // no UI here
})
// in the app
messaging().onMessage((m) => {
  notifee.displayNotification({
    title: m.notification.title,
    body: m.notification.body,
    data: m.notification.data
  })
})
messaging().onNotificationOpenedApp((route) => {
  // from background
  messaging().onTokenRefresh((t) => api.putDevice(t))
})
```

React Native specifics

- @react-native-firebase/messaging: register the background handler early, outside components; an iOS background launch can be **headless** — check isHeadless, render nothing. onMessage shows nothing: use **Notifee**. RNFB's requestPermission is deprecated → react-native-permissions / expo-notifications.
- Expo: expo-notifications + Expo's push service, or raw FCM/APNs tokens.

Server fan-out

Event → queue → workers: look up the user's live tokens, render per locale, send **one message per token** (Admin SDK sendEach/sendEachForMulticast, ≤ 500 per call; sendAll/sendMulticast are deprecated), **topics** for broadcasts. Retry 429/5xx with backoff; delete on UNREGISTERED (404), and on INVALID_ARGUMENT (400) only if the payload is valid (APNs: 410). Idempotency key per event: a retried job must not double-notify.

Traps

- ! Android **notification** message in background: your handler never runs — send **data**.
- ! No deleteToken on logout — the next user gets the old one's pushes.
- ! Data-only to iOS as a sync trigger — throttled, dropped after force-quit.
- ! Xcode build (sandbox APNs) vs TestFlight (production): different tokens.

Remember

"Token per install, rows per user, one send per token, delete on 404 — and know which messages your code never sees."